

21/05/2025

FIFO REALIZAT CU MEMORIE RAM SINCRONA

PROIECT PSN



**UNIVERSITATEA
TEHNICĂ**
DIN CLUJ-NAPOCA

Susman Bogdan

Grupa: r_30214

INDRUMATOR: ING. Ella Timea Seres

Cuprins

1	Specificatii.....	3
2	Proiectare.....	4
2.1	Schema Bloc.....	5
2.2	Unitatea de Control si Unitatea de Executie.....	6
1.2.1	Maparea intrarilor si iesirilor cutiei mari pe cele doua componente UC si UE + Schema bloc a primei descompuneri.....	5
1.2.2	Determinarea resurselor (UE).....	7
2.2.4	Reprezentarea UC prin diagrama de stari (organigrama).....	15
2.2.5	Schema de detaliu a proiectului.....	16
3	Manual de utilizare.....	16
4	Justificarea solutiei alese.....	17
5	Posibilitati de dezvoltari ulteriare.....	18
6	Tutorial.....	19
7	Bibliografie si Webgrafie.....	20

FIFO REALIZAT CU MEMORIE RAM SINCRONA

1 SPECIFICATII

Proiectul consta in realizarea unui modul FIFO (First In First Out) utilizand o memorie RAM sincrona, destinat integrarii în sisteme digitale unde este necesara gestionarea fluxurilor de date cu rate de scriere și citire variabile. Modulul va fi utilizat ca un tampon de date (buffer) între doua blocuri logice care functioneaza cu frecvente diferite, sau au momente diferite de acces la date.

FIFO-ul va respecta următoarele specificatii functionale:

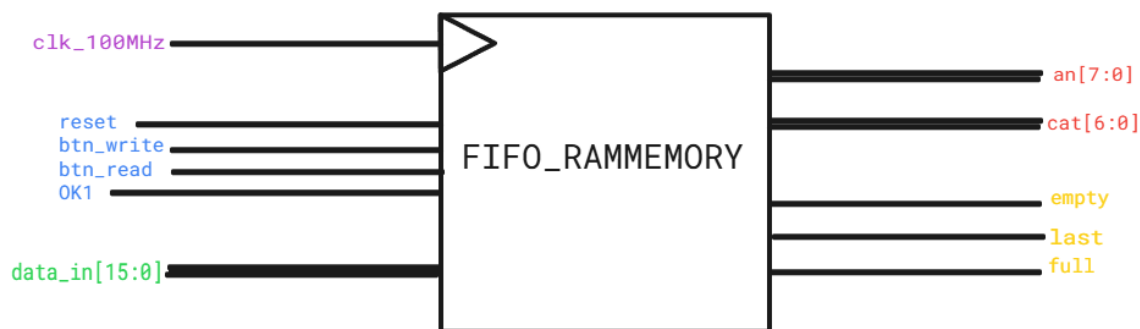
- **Capacitate fixa** de stocare a unui numar prestabilit de cuvinte.
- **Doua interfețe de acces**, complet separate:
 - Una pentru **scriere** (write enable, data in, write clock);
 - Alta pentru **citire** (read enable, read clock);
- Utilizarea unei **memorii RAM sincrone** pentru stocarea efectiva a datelor.
- Implementarea unui **pointer de scriere** și a unui **pointer de citire**, gestionati de o **unitate de control**.
- Semnale de stare:
 - **empty**: indica faptul ca FIFO-ul este gol si nu exista date de citit;
 - **full**: indica faptul ca FIFO-ul este plin si nu se mai pot scrie date;
 - **last**: semnal care indica citirea ultimei valori disponibile inainte ca FIFO-ul sa devina gol (optional, dar util pentru fluxuri seriale de date).
- Daca se incearca o operatie de scriere în timp ce FIFO-ul este plin, sau o citire când FIFO-ul este gol, operatia este ignorata (comportament safe).
- Toate operatiile de acces (scriere/citire) trebuie sa fie **sincrone cu semnalele de ceas** corespunzatoare.

Funcționalitatea modului FIFO va fi realizată într-un mod cât mai eficient din punct de vedere al resurselor hardware, utilizând componente de bază precum Memoria RAM, Multiplexoare, Bistabile de tip D, MPG(pentru butoane), BCD 7 segments display si Divizor de frecventa(pentru clock)

Scopul acestui proiect este acela de a demonstra capacitatea de proiectare a unui sistem digital sincronizat, capabil sa gestioneze coerent o memorie de tip RAM pentru implementarea unui FIFO robust, ce poate fi utilizat în diverse aplicatii FPGA.

2 PROIECTARE

2.1 SCHEMA BLOC



Mov -> Clock-ul intern

Rosu -> afisarea pe 7segment display

Albastru -> butoane

Galben -> led-uri pentru avertizare

Verde -> switch-urile pentru datele de intrare

În această etapă a proiectării, este esențială stabilirea completă a semnalelor de intrare și ieșire ale modului FIFO_RAMMEMORY. Aceste semnale trebuie alese nu doar pe baza specificațiilor scrise, ci și ținând cont de funcționalitatea efectivă care se dorește a fi implementată pe placa FPGA (în cazul nostru, Nexys4).

În cazul de față, anumite semnale precum `btn_write`, `btn_read` sau `OK1` **nu sunt menționate direct în cerințele teoretice**, dar devin absolut necesare în practică pentru a permite interacțiunea controlată și sigură cu modulul FIFO:

- **`btn_write`**: declanșează scrierea datelor din `data_in` în FIFO;
- **`btn_read`**: declanșează citirea următorului element disponibil;
- **`OK1`**: confirmă acțiuni precum validarea introducerii sau sincronizarea comenzilor (poate funcționa ca un debounce hardware).

Aceste semnale sunt introduse pentru a permite integrarea practică a FIFO-ului pe placa Nexys4, folosind resursele fizice disponibile:

- **16 switch-uri** (folosite pentru a introduce date binare);
- **5 butoane** (reset, confirmări, citire/scriere);
- **8 SSD (Seven Segment Display)** (pentru afișarea valorilor);
- **16 LED-uri** (pentru debugging vizual sau afișarea stărilor).

Este important ca în acest moment să se simuleze mental toate scenariile de utilizare (use case-uri) ale sistemului:

1. Introducerea unei valori de 16 biți → confirmare prin **OK1** → apăsarea **btn_write**;
2. Citirea unei valori → apăsarea **btn_read**;
3. Tratarea stărilor de **FIFO plin (full)** și **FIFO gol (empty)**;
4. Detectarea momentului în care s-a citit **ultima valoare disponibilă (last)**.

Proiectantul trebuie să fie pregătit să adauge și pe parcurs alte semnale auxiliare dacă apar cerințe funcționale suplimentare sau dacă apar nevoi impuse de interfatarea cu FPGA-ul.

Afișarea va funcționa doar pentru o placă Nexys care are 8 anozii pentru afișare. Dacă spre exemplu am vrea să punem pe o placă Basys-3 cu afișare pe 4 anozii, atunci logica afișării va trebui schimbată.

Astfel, **structura de semnale** nu este rigidă, ci evolutivă, adaptată dinamic pe măsura ce designul progresează.

2.2 UNITATEA DE CONTROL SI DE EXECUTIE

Cuția neagră a sistemului FIFO_RAMMEMORY trebuie descompusă pentru a putea identifica și implementa componentele necesare. Vom aplica o metodă de proiectare **top-down**, în care sistemul este împărțit treptat în blocuri funcționale mai simple, până când ajungem la circuite cunoscute și ușor de implementat. După această fază, vom proceda **bottom-up**, construind sistemul final din componentele identificate.

Primul pas în descompunere este separarea **logicii de control** de **resursele funcționale ale sistemului**:

- **Logica de control** este responsabilă de luarea deciziilor în funcție de starea FIFO-ului și comenzile utilizatorului (scriere, citire, reset etc.).
- **Resursele sistemului** sunt componentele care gestionează datele în sine, adică memoria RAM, pointerii, și semnalele de stare.

Unitatea de Control (U.C.):

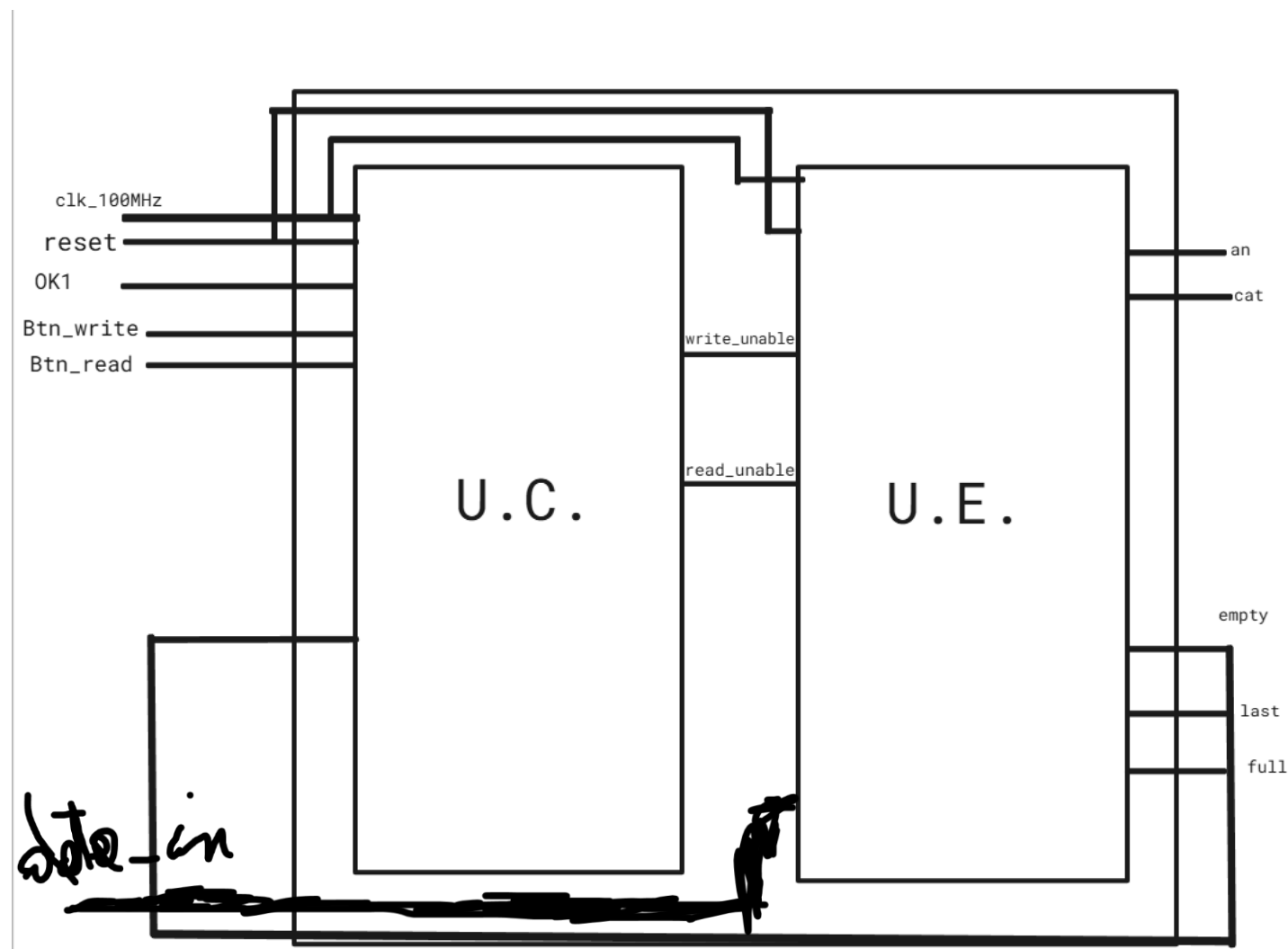
Gestionează semnalele de control `write_enable`, `read_enable`, interpretează semnalele de la butoane (`btn_write`, `btn_read`, `OK1`), verifică stările FIFO (`empty`, `full`) și decide dacă se pot face operații.

Unitatea de Execuție (U.E.):

Conține efectiv memoria FIFO (RAM sincrona), registrele pentru pointerul de scriere și pointerul de citire, comparatoarele de stare (full, empty, last) și logica pentru citire/scriere.

Această separare este esențială pentru un design clar și modular. **Orice algoritm hardware** poate fi abordat astfel: partea de decizie (U.C.) și partea care execută concret acțiunile (U.E.).

1.2.1 Maparea Unitatii de control si a unitatii de executie cu datele de intrare si iesire



Exemplu de implementare top-down → bottom-up:

1. Top-down:

a. FIFO general

↳ U.C. → logică de control (scriere/citire/reset)

↳ U.E. → RAM, pointeri, semnale empty, full

2. Bottom-up:

- a. Se implementează RAM-ul sincron
- b. Se construiesc registrele pointerilor
- c. Se adaugă comparatoarele pentru **empty** și **full**
- d. Se construiește logica de incrementare a pointerilor
- e. Se integrează afișajul și comenzile externe

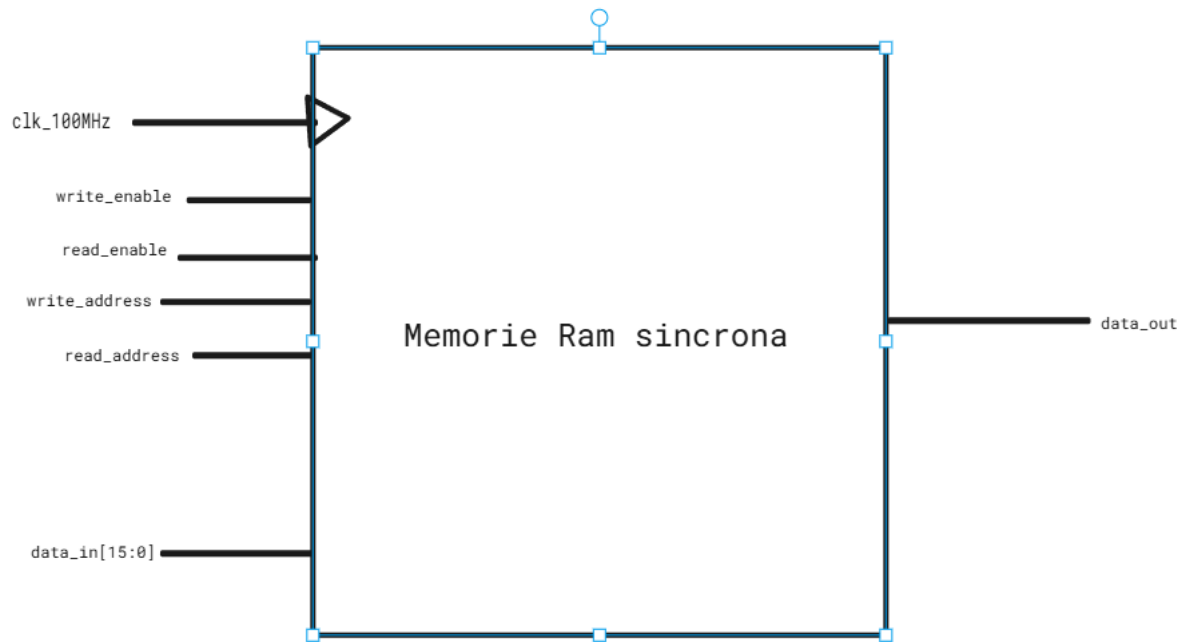
2.2.2 Determinarea resurselor (UE)

Pentru a stabili mai departe legăturile dintre UC și UE trebuie mai întâi să identificăm resursele pe baza cărora luăm decizii sau resursele care devin informații pentru utilizator. Aceste resurse pot să genereze semnale către unitatea de control și pot fi controlate de UC prin semnale de Enable sau Reset. Orice informație pe baza căreia se ia decizii trebuie să vină de la o resursă care generează acea informație și o transmite mai departe UC.

Resursele pot fi circuite simple, care pot fi implementate direct (numarator, registru etc) sau resurse complexe (algoritm de dat rest, algoritm de inmultire, etc). Aceste resurse complexe pot apărea în prima descompunere cu cutii negre cărora trebuie să le stabilim intrări și ieșiri, dar ulterior trebuie descompuse mai departe (de obicei tot în UC și UE) până când ajungem la circuite cunoscute.

Resurse:

1. Memorie Ram Sincrona



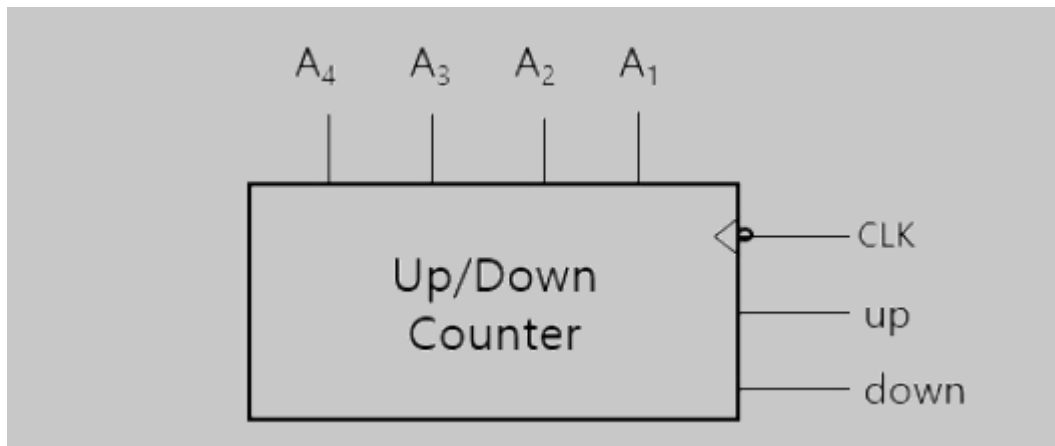
Memoria RAM sincrona este folosita în FIFO pentru **stocarea temporara a datelor** într-un mod eficient, rapid si sincronizat cu ceasul sistemului.

De ce folosim RAM:

- **Acces rapid** (citire/scriere în 1-2 cicluri de ceas).
- **Eficienta hardware:** folosește puține resurse pe FPGA comparativ cu registrele.
- **Scalabilitate:** poți gestiona zeci/sute de cuvinte fara probleme.
- **Sincronizare perfecta** cu clock-ul, ideală pentru fluxuri de date controlate.

RAM-ul este alegerea standard pentru FIFO deoarece oferă **viteza, predictibilitate și consum redus de resurse**.

2. Numarator SUS JOS 4(4 bit up/down counter)



Imaginea de mai sus reprezintă un **numarator sincronizat Up/Down** care corespunde modului utilizat în cod pentru generarea pointerilor FIFO (write_ptr și read_ptr).

În cadrul implementării, acest bloc este responsabil cu **incrementarea** adresei de scriere (când write_enable = '1') sau **decrementarea** în cazul unui contor bidirecțional (deși în FIFO standard folosim doar incrementare).

Semnalul de ceas (CLK) asigură sincronizarea operațiilor, iar ieșirile A₁–A₄ corespund adresei binare curente, transmisă către memoria RAM sincronă pentru scriere sau citire.

Codul sursă pentru Numarator este:

```
entity Numarator_SUS_JOS_4 is Port (
```

```
CE_WR : in STD_LOGIC; CE_RD : in STD_LOGIC;
```

```
CLK : in STD_LOGIC; RESET : in STD_LOGIC;
```

```
Q : out STD_LOGIC_VECTOR(3 downto 0)
```

```
); end Numarator_SUS_JOS_4;
```

```
architecture Behavioral of Numarator_SUS_JOS_4 is signal cnt : unsigned(3 downto 0) := (others => '0');
begin
```

```
process(CLK, RESET)
```

```
begin
```

```
    if RESET = '1' then
```

```
        cnt <= (others => '0'); -- Resetare imediata
```

```
    elsif rising_edge(CLK) then
```

```
        if CE_WR = '1' and CE_RD = '0' then
```

```
            cnt <= cnt + 1;
```

```
        elsif CE_RD = '1' and CE_WR = '0' then
```

```
            cnt <= cnt - 1;
```

```

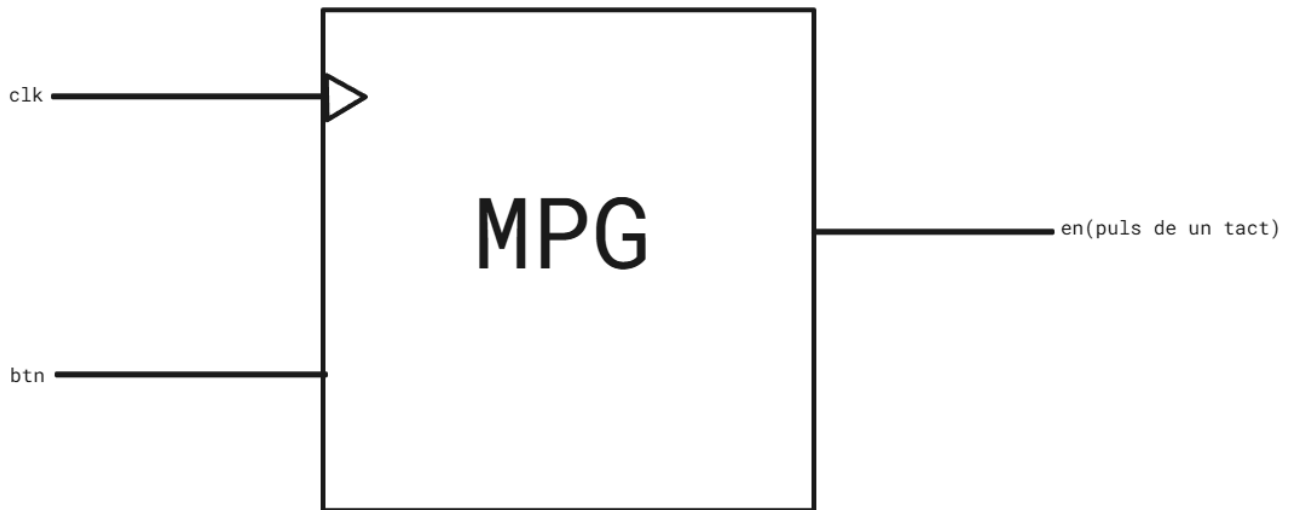
        end if;
    end if;
end process;

Q <= std_logic_vector(cnt);

end Behavioral;

```

3. MPG(pentru butoane)



MPG functioneaza astfel:

- Preia ca intrare semnalul de la buton (**btn**) si semnalul de ceas (**clk**)
- Intern, sincronizeaza semnalul butonului cu ceasul sistemului printr-un bistabil D
- Detecteaza **flancul negativ** (tranzitia de la '1' la '0') si genereaza un puls logic (**en = 1**) pentru exact un ciclu de ceas
- Daca butonul ramane apasat, iesirea **en** nu ramane activa – doar tranzitia initiala este recunoscuta

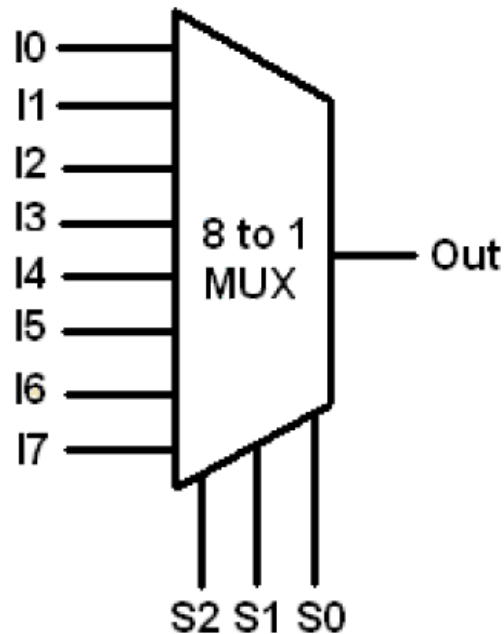
Utilizare:

```
debounce_write: MPG port map(clk => clk_100MHz, btn => btn_write, en => btn_write_db);
```

```
debounce_read: MPG port map(clk => clk_100MHz, btn => btn_read, en => btn_read_db);
```

```
debounce_ok: MPG port map(clk => clk_100MHz, btn => OK1, en => ok_db);
```

4. Multiplexoare (MUX)



În proiect Multiplexoarele sunt folosite pentru mai multe lucruri:

- **Multiplexor adresa RAM (2:1)**

Selectează între `write_ptr` și `read_ptr` pentru a trimite adresa corectă către RAM, în funcție de operația curentă (`write_enable`). Este necesar deoarece RAM-ul are un singur port de adresă, iar FIFO trebuie să facă atât scriere, cât și citire.

- **Multiplexor afisaj 7 segmente (8:1)**

Selectează pe rând câte un grup de 2 biți (`part1` - `part8`) din `valoare_afisata`, pe baza semnalului `decision`. Acesta permite afișarea dinamică a tuturor cifrelor pe un display multiplexat.

- **Multiplexor selecție valoare afisată (2:1)**

Alege între `data_in` și ultima valoare citită, în funcție de semnalul `OK1`. Astfel, utilizatorul poate opta între afișarea datelor introduse sau a celor extrase din FIFO.

Unde am folosit Multiplexoare?(linii de cod)

```
part1<=valoare_displayed(1 downto 0); part2<=valoare_displayed(3 downto 2);
part3<=valoare_displayed(5 downto 4); part4<=valoare_displayed(7 downto 6);
part5<=valoare_displayed(9 downto 8); part6<=valoare_displayed(11 downto 10);
part7<=valoare_displayed(13 downto 12); part8<=valoare_displayed(15 downto 14);
```

```
process(clk_100MHz) begin if rising_edge(clk_100MHz) then case decision is when "000" =>
digit <= part1; an <= "11111110"; when "001" => digit <= part2; an <= "11111101"; when "010"
=> digit <= part3; an <= "11111011"; when "011" => digit <= part4; an <= "11110111"; when
"100" => digit <= part5; an <= "11101111"; when others => digit <= part6; an <= "11011111";
```

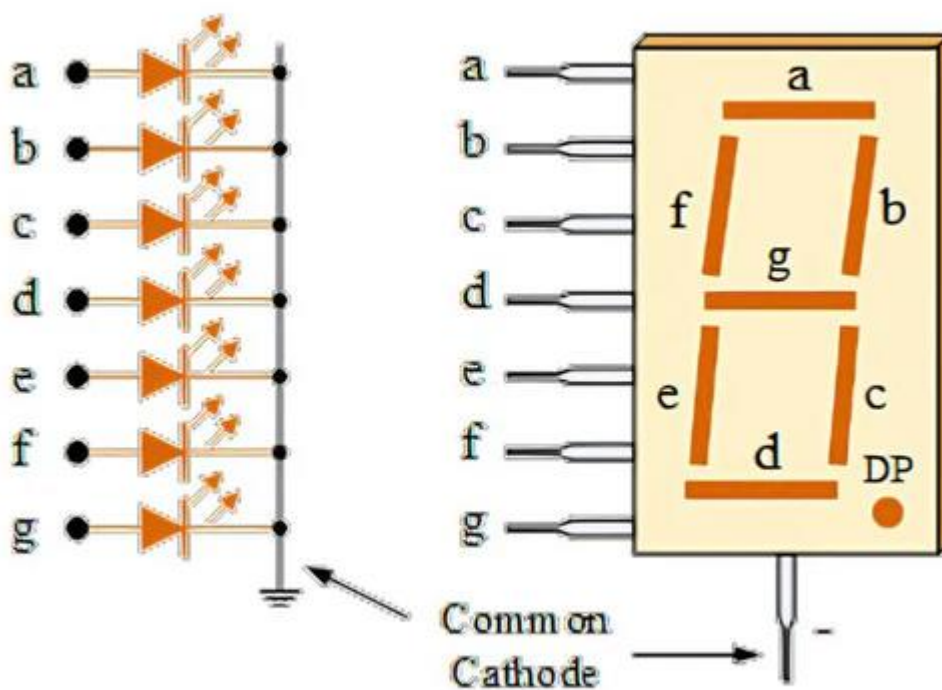
```
end case;  
end if;
```

```
end process;
```

(cod pentru crearea bazei rezultatelor pe anozii.)

Cum arata anozii si catozii:

Sunt in numar de 8 pe o placuta Nexys de aceea am ales baza 4 pentru afisare (toti catozii si anozii sa fie folositi)



Dupa aceea avem aceasta parte de cod pentru afisare(abcdefg):

```
process(digit)
```

```
begin
```

```
case digit is
```

```
when "0000" => cat <= "1000000"; -- 0
```

```
when "0001" => cat <= "1111001"; -- 1
```

```
when "0010" => cat <= "0100100"; -- 2
```

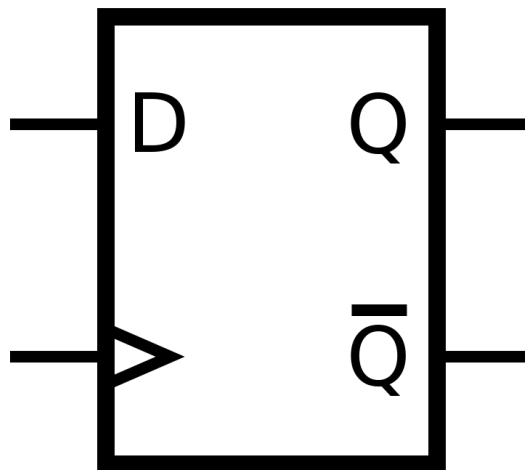
```
when "0011" => cat <= "0110000"; -- 3
```

```
when others => cat <= "1111111";-- dezactivat
```

```
End case;
```

```
End process;
```

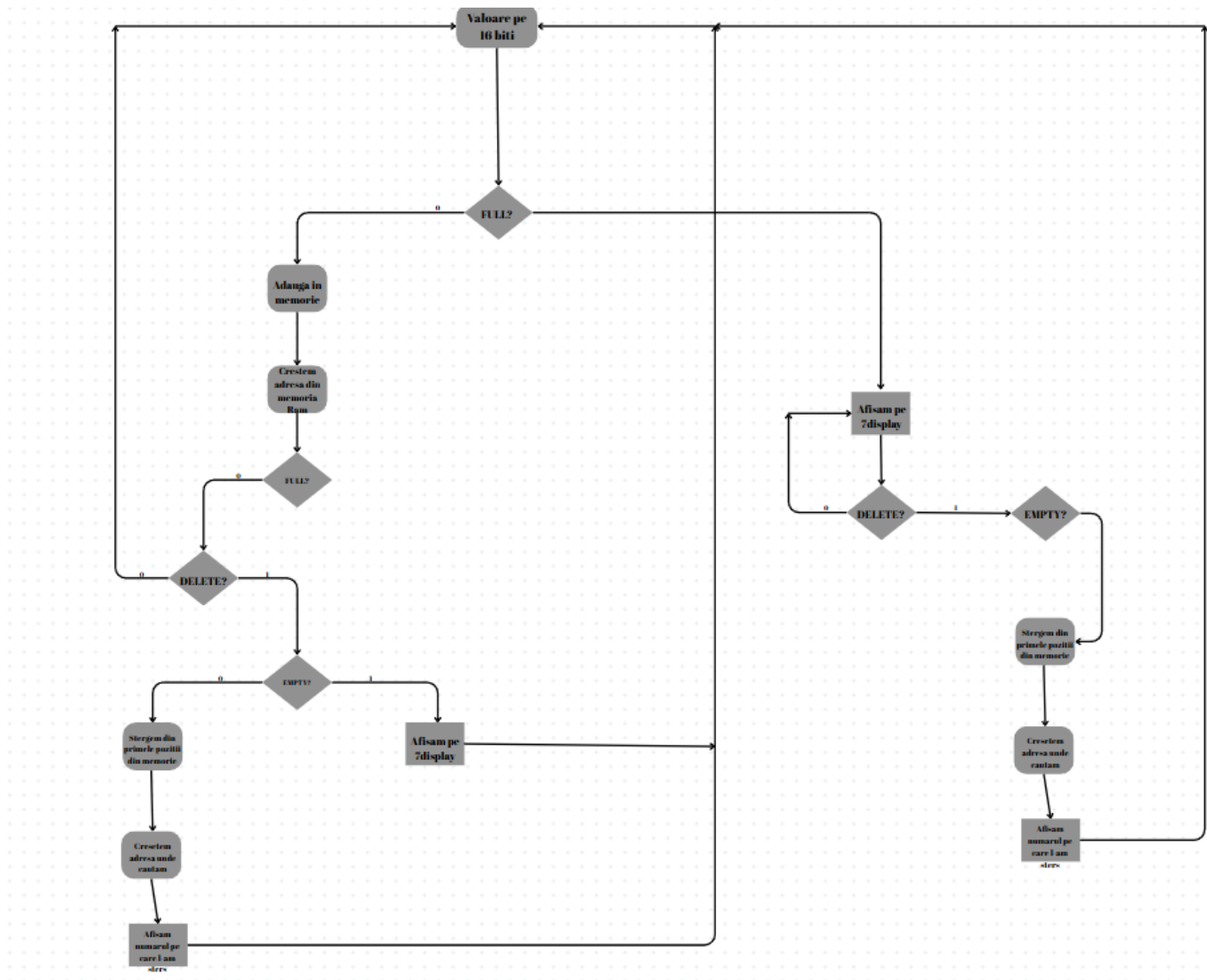
5. Bistabile D (flip-flop)



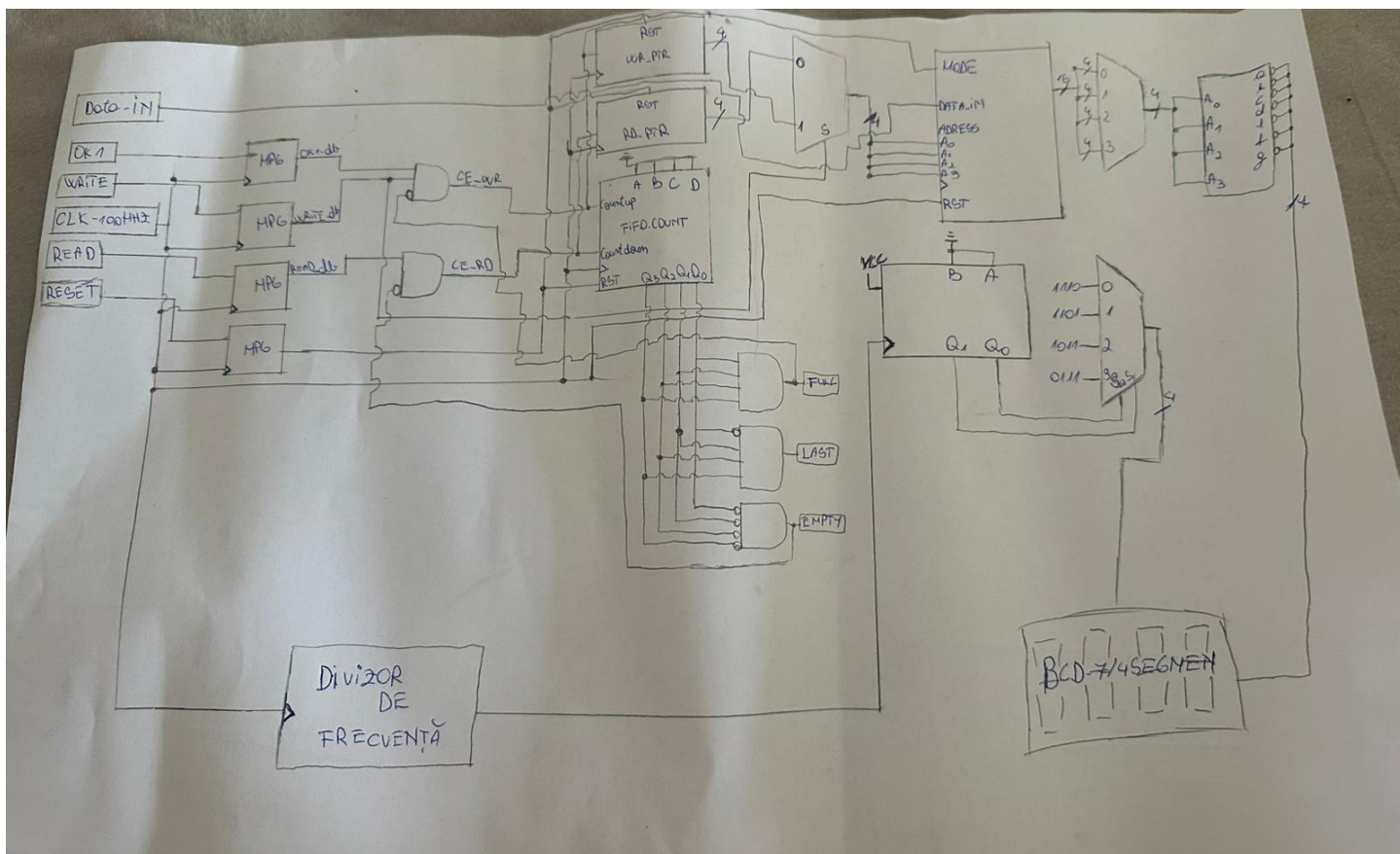
În cadrul procesului de afișare pe display-ul cu 7 segmente, valorile binare extrase din semnalul *valoare_afisata* sunt împărțite în grupuri de câte 2 biți (*part1* – *part8*) și trecute printr-o zonă logică echivalentă cu bistabile de tip D, sincronizate cu ceasul *clk_100MHz*.

Această stabilizare este esențială pentru ca multiplexorul de selecție (*decision*) să opereze corect. La fiecare ciclu de ceas, doar una dintre cele 8 perechi de biți este afișată, iar bistabilele D asigură că valorile nu se schimbă în timpul comutării rapide.

2.2.3 Reprezentarea UC prin diagrama de stari (organigrama)



2.2.4 Schema in detaliu a proiectului



3. MANUAL DE UTILIZARE ȘI ÎNTREȚINERE

Utilizare

Pentru a utiliza corect aplicația FIFO realizată cu memorie RAM sincronă, utilizatorul trebuie să respecte pașii următori:

1. Resetare sistem

Apasă butonul reset pentru a inițializa pointerii, memoria RAM și afișajele.

2. Introducerea unei valori

- Utilizează cele 16 switch-uri de pe placa Nexys4 pentru a forma un cuvânt binar de 16 biți.
- Apasă butonul OK1 pentru a confirma valoarea introdusă. Valoarea va fi afișată temporar pe display.

3. Scrierea în FIFO

- Apasă btn_write. Dacă FIFO nu este plin ($full = 0$), valoarea introdusă se scrie în memorie.

4. Citirea din FIFO

- Apasă btn_read. Dacă FIFO nu este gol ($empty = 0$), următoarea valoare disponibilă va fi citită și afișată pe SSD.

5. Monitorizare stări

- LED-urile sau alte semnale pot indica stările empty, full și last.
- Afișajul arată valoarea curentă activă din FIFO.

Întreținere

- Sistemul este hardware sincron, nu necesita intretinere software.
- Pentru depanare se recomanda:
 - Verificarea semnalelor `write_enable`, `read_enable` in simulare.
 - Resetarea completa în caz de comportament neasteptat.
 - Verificarea fizica a switch-urilor si butoanelor pentru contact imperfect.

4. JUSTIFICAREA SOLUȚIEI ALESE

Implementarea FIFO-ului folosind o memorie RAM sincrona a fost aleasa pentru eficienta și scalabilitate hardware. Iata principalele decizii și justificarile lor:

Alegerea RAM sincronă vs. registre simple

- RAM sincrona permite stocarea unui numar mai mare de valori cu un consum minim de resurse FPGA.
- Registrele individuale (flip-flopuri) ar fi consumat mai multă logică și nu ar fi fost scalabile pentru $DEPTH > 8$.

Alegerea MPG în loc de debounce clasic

- Modulul MPG este mai eficient decât debounce cu counter deoarece produce un impuls sincron curat, fără logică redundantă.
- Este perfect integrabil în logica sincrona, evitând glitch-uri la citire/scriere.

Afișarea cu multiplexare vs. afisaj static

- Multiplexarea celor 8 cifre reduce drastic numărul de pini necesari pentru afisare.
- Creează un efect vizual fluid și permite afisarea completă a valorii FIFO pe 7 segmente.

5. POSIBILITĂȚI DE DEZVOLTĂRI ULTERIOARE

Pe baza arhitecturii actuale, proiectul poate fi extins în mai multe direcții. Iată două exemple concrete:

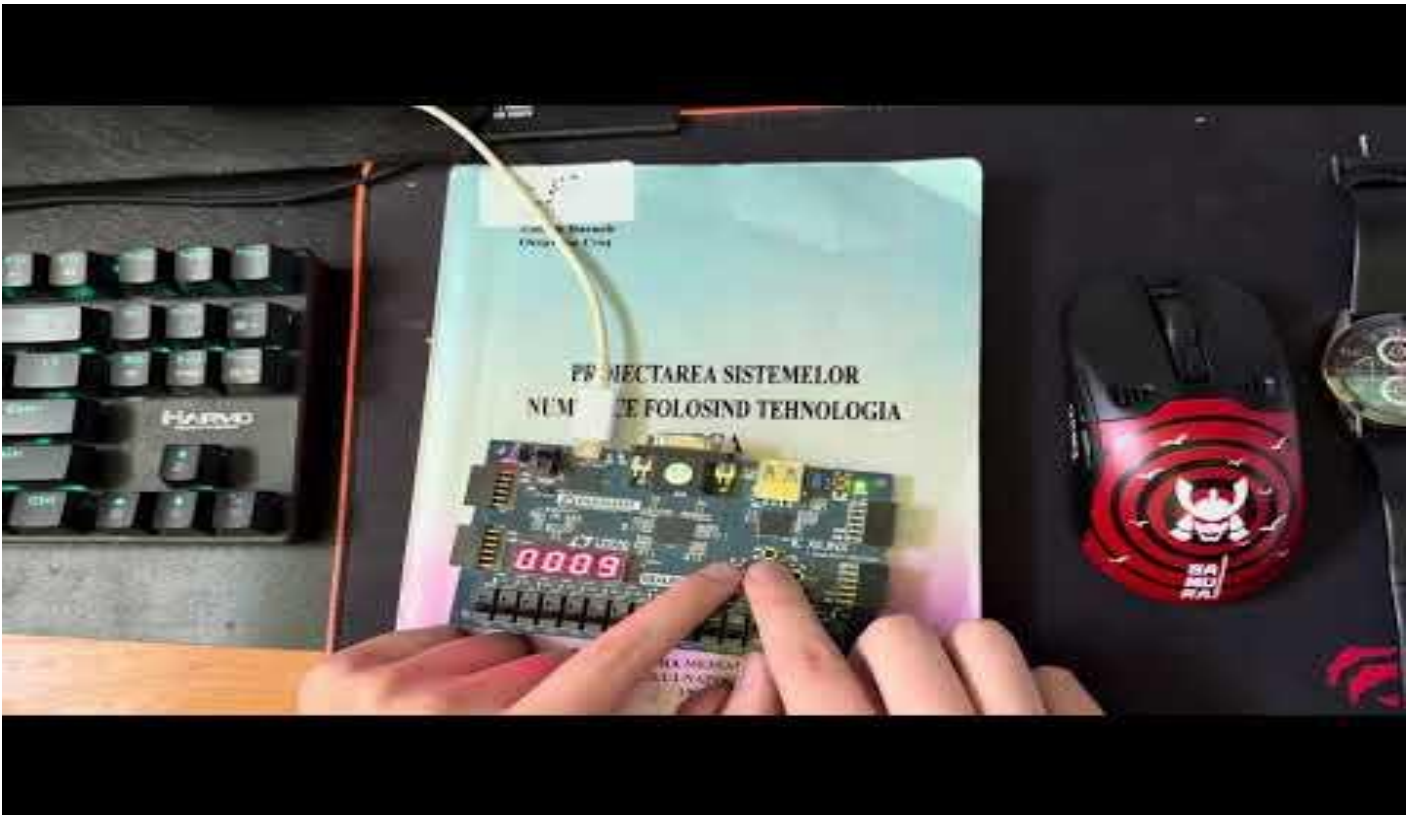
1. FIFO asincron

- Se poate înlocui RAM-ul cu o versiune dual-port, cu ceasuri independente pentru scriere și citire.
- Se adaugă logica de sincronizare a pointerilor și detecție de full/empty în mod asincron.

2. Suport pentru UART sau SPI

- Se poate conecta FIFO-ul la o interfață UART pentru a trimite și primi date seriale.
- `data_in` și `data_out` pot fi legate la transceiver-ul serial, astfel FIFO devine un buffer pentru transmisii UART/SPI.

6. TUTORIAL DE FOLOSIRE



7. BIBLIOGRAFIE SI WEBGRAFIE

- Webgrafie

<https://www.youtube.com/watch?v=jxzZJOawjBo>

https://en.wikipedia.org/wiki/Double_dabble

<https://youtu.be/7EwYQlkeTYE?si=XupaPp7QVQX-gDND>

<https://www.electronics-tutorials.ws/blog/7-segment-display-tutorial.html>

-Bibliografie

“Proiectarea sistemelor numerice folosind tehnologia FPGA” de S. Nedevschi, Z. Baruch, O Cret (disponibila pentru imprumut la Biblioteca UTCN), la pagina 238.